

claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_87b22fa4-6ac\agent-  
a5fa5ae610b8a8a42.jsonl`  
- SHA-256: `66ed8956ed79d1e4ec59f18ec5fc1366164133891327009836e15519edee29da`  
- Source modified: `2026-06-21T10:47:46+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_87b22fa4-6ac`  
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`
```

Transcript

1. user (2026-06-21T10:45:15.848Z)

Review the COMPUTE_DECOUPLED_SERVING **M6** change just committed on branch feat/serving-m6-cloudrun-dispatch (HEAD). Inspect with: git diff origin/master...HEAD , and read the full files:

```
- backend/compute/base.py (ComputeBackend ABC + ComputeJobSpec + ComputeResult)  
- backend/compute/cloud_run.py (CloudRunCompute: dispatch a Cloud Run Job via injectable  
CloudRunJobClient, then bucket-poll a done-marker via execution_policy.poll_until + the storage  
layer; GoogleCloudRunJobClient = real client, lazy google-cloud-run)  
- backend/compute/__init__.py (get_compute_backend(config) ? backend only for  
compute_target=cloud_run, else None = run inline)  
- backend/worker/__main__.py (cmd_infer: dispatch branch for cloud_run + _dispatch_remote;  
--done-uri + _write_done_marker on success)  
- deploy/cloudrun/Dockerfile + README.md (the L4 image)  
- tests/test_compute_backend.py  
CONTRACT: DEFAULT-OFF ? only compute_target=cloud_run dispatches; ""/local_gpu/cpu_mock run the  
worker's EXISTING in-process path BYTE-IDENTICALLY. The dispatched container must run INLINE  
(never re-dispatch) via forced compute_target=local_gpu+native. Completion = a bucket done-  
marker (not the Cloud Run API status), D16. The real client is owner-verified on M7 (CI uses a  
fake). Report ONLY real, grounded issues with file:line. Empty findings array if a lens finds  
nothing real ? do not invent nits.
```

LENS = INTEGRATION / HYGIENE / CONVENTIONS. (1) Circular imports: backend.compute imports backend.storage + execution_policy; backend.worker imports backend.compute; backend.compute.__init__ lazy-imports cloud_run. Any import cycle or eager heavy import? Is google-cloud-run truly only imported when cloud_run is selected (lazy)? (2) The storage usage in cloud_run.py (_read_marker/_read_json via parse_uri + get_storage + fetch) ? correct for both local:// (tests) and gs:// (prod)? Does it pass storage_config through? (3) Does ComputeJobSpec config_overrides being a tuple (frozen) interact correctly with the worker --set list? (4) Naming/consistency with backend/storage (factory shape, lazy _backend_class idiom). (5) The Dockerfile: any obvious build-breaker (COPY paths exist? pyproject install? the WASB env steps faithful to gpu_setup.sh?). (6) mypy/ruff issues the CI would catch (Any returns, Optional, the run_v2 type: ignore). Report grounded issues only.

2. assistant (2026-06-21T10:45:18.828Z)

I'll review the M6 change on branch feat/serving-m6-cloudrun-dispatch. Let me start by pulling the latest and inspecting the diff.

3. user (2026-06-21T10:45:22.370Z)

```
feat/serving-m6-cloudrun-dispatch  
2266e91 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)  
b5af66e feat(serving): M5 ? truly-local profile per-profile startup checks + L4 profile-parity  
test (default-OFF) (#286)  
fefa412 docs(serving): resolve §8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)
```

4. user (2026-06-21T10:45:22.572Z)

2266e91 feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)

5. assistant (2026-06-21T10:45:24.737Z)

Let me look at the diff and read all the target files in parallel.

6. user (2026-06-21T10:45:25.857Z)

```
backend/compute/__init__.py | 61 ++++++++
backend/compute/base.py     | 52 ++++++++
backend/compute/cloud_run.py | 141 ++++++++
backend/worker/__main__.py   | 51 ++++++++
deploy/cloudrun/Dockerfile   | 61 ++++++++
deploy/cloudrun/README.md    | 59 ++++++++
tests/test_compute_backend.py | 222 ++++++++
7 files changed, 647 insertions(+)
```

7. user (2026-06-21T10:45:26.618Z)

```
1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"cpu"`` / ``"local_gpu"`` / ``"cpu_mock"``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str                                # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str                                  # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
```

```

36         (0 ok / 2 validation-or-config / 3 segmenter-failure). ``video_id`` is known once
the job's
37         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
38
39         returncode: int
40         outputs_uri: str = ""
41         video_id: str = ""
42         report: dict = field(default_factory=dict)
43
44
45     class ComputeBackend(ABC):
46         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
47         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
48         execution identically."""
49
50         @abstractmethod
51         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
52             """Run ``spec`` to completion and return its ``ComputeResult``."""
53

```

8. user (2026-06-21T10:45:27.070Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12            ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14        The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15        so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17        The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18        with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19        owner-verified on the M7 real run.
20        """
21
22        from __future__ import annotations
23
24        import json
25        import logging
26        import uuid
27        from abc import ABC, abstractmethod
28        from typing import Any, List, Optional
29
30        from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult

```

```

31     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
32     from backend.storage import fetch, parse_uri
33     from backend.storage import get_storage
34
35     LOG = logging.getLogger("compute.cloud_run")
36
37     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
38     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
39     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
40         "deployment.compute_target=local_gpu",
41         "indexing.detector_impl=native",
42     )
43
44
45     class CloudRunJobClient(ABC):
46         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
47
48         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
49         overrides the job's container ``args`` for this execution and starts it."""
50
51         @abstractmethod
52         def run_job(self, job_name: str, argv: List[str], *, region: str,
53                     project: Optional[str] = None) -> str:
54             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
55             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
56
57
58     class GoogleCloudRunJobClient(CloudRunJobClient):
59         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
60         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
61
62         def run_job(self, job_name: str, argv: List[str], *, region: str,
63                     project: Optional[str] = None) -> str:
64             try:
65                 from google.cloud import run_v2 # type: ignore[attr-defined]
66             except Exception as exc: # pragma: no cover - real SDK not present in CI
67                 raise RuntimeError(
68                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
69                     "control plane (it is intentionally NOT a CI/test dependency).")
70             ) from exc
71             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
72             name = client.job_path(project, region, job_name)
73             overrides = run_v2.RunJobRequest.Overrides(
74                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
75             )
76             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
77             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
78
79
80     class CloudRunCompute(ComputeBackend):
81         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

```

```

82
83     def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
84                   project: Optional[str] = None, storage_config: Optional[dict] = None,
85                   policy: ExecutionPolicy = DEFAULT_POLICY,
86                   token: Optional[str] = None,
87                   container_overrides: Optional[tuple[str, ...]] = None) -> None:
88         self._client = run_client
89         self._job_name = job_name
90         self._region = region
91         self._project = project
92         self._storage_config = storage_config or {}
93         self._policy = policy
94         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
95         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
96         self._token = token
97         self._container_overrides = (
98             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
99         )
100
101     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
102         out_root = spec.out_uri.rstrip("/")
103         token = self._token or uuid.uuid4().hex
104         done_uri = f"{out_root}/_dispatch/{token}.done"
105         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
106                           "--done-uri", done_uri]
107         if spec.segmenter:
108             argv += ["--segmenter", spec.segmenter]
109         for kv in (*spec.config_overrides, *self._container_overrides):
110             argv += ["--set", kv]
111
112         execution = self._client.run_job(self._job_name, argv, region=self._region,
113                                         project=self._project)
114         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
115                 self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
118                             label="cloud-run-infer", logger=LOG)
119         video_id = str(marker.get("video_id", ""))
120         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
121         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
122         rc = int(marker.get("returncode", 0))
123         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
124         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
125
126     # --- storage-backed completion signal (bucket-poll) ---
127     def _read_marker(self, done_uri: str) -> Optional[dict]:
128         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
129         ref = parse_uri(done_uri)
130         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
131             return None
132         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
133
134     def _read_json(self, uri: str) -> dict:
135         try:
136             local = fetch(uri, config=self._storage_config)
137             with open(local, "r", encoding="utf-8") as f:
138                 data: Any = json.load(f)

```

```

139         return data if isinstance(data, dict) else {}
140     except Exception:
141         return {}
142

```

9. user (2026-06-21T10:45:27.293Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5     ``None``
6     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
8     ``cpu_mock`` nothing
9     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
10    """
11
12    from __future__ import annotations
13
14    import os
15    from typing import Any, Optional
16
17    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20    __all__ = [
21        "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
22    ]
23
24    def _storage_dict(config: Any) -> dict:
25        storage = getattr(config, "storage", None)
26        if storage is None:
27            return {}
28        if hasattr(storage, "model_dump"):
29            return storage.model_dump() # type: ignore[no-any-return]
30        return dict(storage) if isinstance(storage, dict) else {}
31
32    def get_compute_backend(
33        config: Any,
34        *,
35        run_client: Optional[Any] = None,
36        policy: ExecutionPolicy = DEFAULT_POLICY,
37        token: Optional[str] = None,
38    ) -> Optional[ComputeBackend]:
39        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40        in-process (the default-OFF path).
41
42        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46        """
47        target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48        if target != "cloud_run":
49            return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

```

```

50
51     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57     client = run_client if run_client is not None else GoogleCloudRunJobClient()
58     return CloudRunCompute(
59         run_client=client, job_name=job_name, region=region, project=project,
60         storage_config=_storage_dict(config), policy=policy, token=token,
61     )
62

```

10. assistant (2026-06-21T10:45:29.624Z)

Now let me read the worker, the storage layer, execution_policy, and the remaining files.

11. user (2026-06-21T10:45:31.054Z)

```

1     """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3     All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
simulates the
4     worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
dispatch?bucket-poll
5     ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
``compute_target=cloud_run``
6     selects a backend; everything else returns ``None`` (= run the worker's existing in-
process path).
7     """
8     import json
9     import os
10
11     from backend.compute import ComputeJobSpec, get_compute_backend
12     from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
13     from backend.config import AppConfig
14     from backend.infrastructure.execution_policy import ExecutionPolicy
15
16     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
poll hits).
17     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
18
19
20     class _FakeRunClient(CloudRunJobClient):
21         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
local bucket
22         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
immediately."""
23
24         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
25             self.calls: list[dict] = []
26             self.video_id, self.segment_count, self.returncode = video_id, segment_count,
returncode
27
28         def run_job(self, job_name, argv, *, region, project=None):
29             self.calls.append({"job": job_name, "argv": list(argv), "region": region,
"project": project})
30             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
31             done_uri = argv[argv.index("--done-uri") + 1]
32             outputs = os.path.join(out_uri, "outputs", self.video_id)
33             os.makedirs(outputs, exist_ok=True)
34             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
35                 json.dump({"video_id": self.video_id, "segment_count": self.segment_count,

```

```

36         "status": "success"}, f)
37     os.makedirs(os.path.dirname(done_uri), exist_ok=True)
38     with open(done_uri, "w", encoding="utf-8") as f:
39         json.dump({"video_id": self.video_id, "returncode": self.returncode,
40                 "segment_count": self.segment_count, "status": "success"}, f)
41     return "exec-123"
42
43
44     # ----- factory (default-
OFF)
45
46     def _cfg(compute_target="", storage_backend="local"):
47         return AppConfig.model_validate({
48             "deployment": {"compute_target": compute_target},
49             "storage": {"backend": storage_backend},
50         })
51
52
53     def test_factory_is_none_for_local_targets():
54         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
55         assert get_compute_backend(_cfg("")) is None
56         assert get_compute_backend(_cfg("local_gpu")) is None
57         assert get_compute_backend(_cfg("cpu_mock")) is None
58
59
60     def test_factory_returns_cloud_run_for_cloud_target():
61         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
62         assert isinstance(backend, CloudRunCompute)
63
64
65     # ----- CloudRunCompute dispatch
+ poll
66
67     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
68         client = _FakeRunClient(video_id="abc123", segment_count=3)
69         backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-
south1",
70                                 project="proj", storage_config={}, policy=_FAST,
token="tok")
71         out = str(tmp_path / "bucket")
72         result = backend.run_infer(ComputeJobSpec(
73             video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))
74
75         assert result.returncode == 0
76         assert result.video_id == "abc123"
77         assert result.outputs_uri == f"{out}/outputs/abc123/"
78         assert result.report.get("segment_count") == 3
79         # exactly one Cloud Run Job execution was triggered, in the right region/project
80         assert len(client.calls) == 1
81         assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] ==
"asia-south1"
82
83
84     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
85         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
86         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
87         client = _FakeRunClient()
88         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST,
token="tok")
89         backend.run_infer(ComputeJobSpec(
90             video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
91             config_overrides={"sport": "tennis"}))

```



```

92
93     argv = client.calls[0]["argv"]
94     assert argv[0] == "infer"
95     assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
96     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
97     # the container-inline overrides come last (win) and the user override is carried
through
98     joined = " ".join(argv)
99     assert "deployment.compute_target=local_gpu" in joined
100    assert "indexing.detector_impl=native" in joined
101    assert "sport=tennis" in joined
102    # user override precedes the inline-forcing overrides in --set order
103    assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")
104
105
106    def test_unique_token_per_call_when_not_injected(tmp_path):
107        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
108        client = _FakeRunClient()
109        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
110        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
111        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
112        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
113        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
114        assert done1 != done2
115
116
117    # ----- worker wiring: dispatch branch +
done-marker
118
119    def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
120        """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
121        from backend.compute.base import ComputeResult
122        from backend.worker import __main__ as wmain
123
124        seen = {}
125
126        class _FakeBackend:
127            def run_infer(self, spec):
128                seen["spec"] = spec
129                return ComputeResult(returncode=0, video_id="vidX",
outputs_uri="gs://b/outputs/vidX/")
130
131        monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
132        cfg = tmp_path / "c.json"
133        cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gs"}}}')
134        rc = wmain.cmd_infer(wmain.build_parser().parse_args([
135            "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
136            "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
137        assert rc == 0
138        assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
139        assert seen["spec"].segmenter == "wasb_hybrid"
140
141
142    def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
143        """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
144        remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
145        from backend.worker import __main__ as wmain

```

```

146
147 class _OK:
148     passed = True
149     validator_name = "fake"
150     message = "ok"
151
152     def model_dump(self):
153         return {"validator_name": "fake", "passed": True, "message": "ok"}
154
155 monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
156                     lambda path, cfg: ([_OK()], {}))
157 monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
158
159 monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
160                     lambda p: 30.0)
161
162 cfg = tmp_path / "c.json"
163 cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
164 True},
165                             "output": {"output_dir": str(tmp_path / "o")}}))
166
167 video = tmp_path / "in.mp4"
168 video.write_bytes(b"FAKEVIDEO")
169 out = tmp_path / "bucket"
170 done = str(out / "_dispatch" / "tok.done")
171
172 rc = wmain.cmd_infer(wmain.build_parser().parse_args([
173     "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
174     "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
175 done]))
176
177 assert rc == 0
178 marker = json.loads(open(done, encoding="utf-8").read())
179 assert marker["returncode"] == 0 and marker["status"] == "success"
180 assert marker["video_id"] and marker["segment_count"] >= 1
181
182 def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
183     """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
184     behaviour)."""
185     from backend.worker import __main__ as wmain
186
187     class _OK:
188         passed = True
189         validator_name = "fake"
190         message = "ok"
191
192         def model_dump(self):
193             return {"validator_name": "fake", "passed": True, "message": "ok"}
194
195     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
196                         lambda path, cfg: ([_OK()], {}))
197     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
198
199 monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
200                     lambda p: 30.0)
201
202 cfg = tmp_path / "c.json"
203 cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
204 True},
205                             "output": {"output_dir": str(tmp_path / "o")}}))
206
207 video = tmp_path / "in.mp4"
208 video.write_bytes(b"FAKEVIDEO")
209 out = tmp_path / "bucket"
210 rc = wmain.cmd_infer(wmain.build_parser().parse_args([
211     "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
212     "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
213
214 assert rc == 0

```

```

205         assert not (out / "_dispatch").exists() # no marker dir when --done-uri is omitted
206
207
208     # ----- Dockerfile structural guard
209     (no build)
210
211     def test_dockerfile_has_required_directives():
212         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
213         structurally:
214         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
215         worker entrypoint."""
216         from pathlib import Path
217
218         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
219         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
220         assert "ffmpeg" in text # stitch/decode
221         assert "fonts-dejavu" in text # the watermark filter font
222         assert "torch==1.11.0+cu113" in text # the verified WASB stack
223         assert "WASB-SBDT" in text # the shuttle detector repo
224         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
225         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text

```

12. user (2026-06-21T10:45:31.110Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ... # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs
11    are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13    key) ?
14    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17    CLI uses
18    (validator registry ? segmenter ? report), never a forked pipeline.
19    """
20
21    from __future__ import annotations
22
23    import argparse
24    import csv
25    import json
26    import logging
27    import os
28    import sys
29    import tempfile
30    from typing import Any, List
31
32    from backend import storage
33    from backend.compute import ComputeJobSpec, get_compute_backend
34    from backend.config import (
35        StartupCheckError,
36        enforce_startup_checks,
37        load_config,
38        probe_cuda_available,
39    )
40
41    from backend.infrastructure.database import Database
42    from backend.pipeline.segmenters.base import segmenter_registry

```

```

37     from backend.pipeline.validators.base import registry as validator_registry
38     from backend.reporting import emit_indexer_report
39     from backend.results import Failure
40     from backend.utils.hashing import compute_video_id
41
42     logger = logging.getLogger("backend.worker")
43
44
45     def _coerce(v: str) -> Any:
46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
47         str."""
48         low = v.lower()
49         if low in ("true", "false"):
50             return low == "true"
51         for cast in (int, float):
52             try:
53                 return cast(v)
54             except ValueError:
55                 pass
56         return v
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
60         indexing.skip_ai_handoff=true)."""
61         for kv in sets or []:
62             key, sep, val = kv.partition("=")
63             if not sep:
64                 raise ValueError(f"--set expects k=v, got {kv!r}")
65             parts = key.split(".")
66             obj = config
67             for p in parts[:-1]:
68                 obj = getattr(obj, p)
69             setattr(obj, parts[-1], _coerce(val))
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
73         friendly
74         artifact (same schema as the rally-annotator labels)."""
75         with open(path, "w", newline="") as f:
76             w = csv.writer(f)
77             w.writerow(["start", "end", "shot_count", "ending_reason"])
78             for s in segments:
79                 w.writerow([
80                     f'{float(s.get("start_time", 0.0)):.3f}',
81                     f'{float(s.get("end_time", 0.0)):.3f}',
82                     s.get("shot_count", ""),
83                     s.get("ending_reason", s.get("shot_count_confidence", "")),
84                 ])
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
87     None:
88         with open(path, "w") as f:
89             json.dump({
90                 "video_id": video_id,
91                 "status": status,
92                 "segment_count": len(segments),
93                 "segments": [{"start": float(s.get("start_time", 0.0)),
94                             "end": float(s.get("end_time", 0.0))} for s in segments],
95             }, f, indent=2)
96
97     def _run_startup_checks(config: Any) -> bool:

```

```

98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch."""
120         spec = ComputeJobSpec(
121             video_uri=args.video_uri, out_uri=args.out_uri,
122             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
123         )
124         result = compute.run_infer(spec)
125         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
126                     result.video_id, result.returncode, result.outputs_uri)
127         return result.returncode
128
129
130     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
131                             status: str, storage_cfg: dict, scratch: str) -> None:
132         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
133         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
134         try:
135             marker = {"video_id": video_id, "returncode": returncode,
136                      "segment_count": segment_count, "status": status}
137             local = os.path.join(scratch, "_done.json")
138             with open(local, "w", encoding="utf-8") as f:
139                 json.dump(marker, f)
140             storage.put(local, done_uri, config=storage_cfg)
141             logger.info("[worker] wrote completion marker -> %s", done_uri)
142         except Exception as e: # never fail a good run because the marker push hiccuped
143             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
144
145
146     def cmd_infer(args: argparse.Namespace) -> int:
147         config = load_config(args.config) if args.config else load_config()
148         _apply_overrides(config, args.set)
149         if not _run_startup_checks(config):

```

```

150         return 2
151
152     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
153     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
154     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
155     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
156     compute = get_compute_backend(config)
157     if compute is not None:
158         return _dispatch_remote(compute, args)
159
160     storage_cfg = config.storage.model_dump()
161
162     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
163     os.makedirs(scratch, exist_ok=True)
164     config.output.output_dir = scratch
165
166     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
167     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
168     print(f"[worker] pulled {args.video_uri} -> {local_video}")
169
170     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
171     video_id = compute_video_id(local_video)
172
173     # 3. VALIDATE (same registry as the main CLI).
174     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
175     failures = [r for r in val_results if not r.passed]
176     db = Database(os.path.join(scratch, config.output.db_name))
177     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
178
179     segments: List[dict] = []
180     segmenter_actual = None
181     segmentation_ran = False
182     status = "failed"
183     try:
184         if failures:
185             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
186             return 2
187
188             seg_name = args.segmenter or config.indexing.default_segmenter
189             segmenter_cls = segmenter_registry.get(seg_name)
190             if not segmenter_cls:
191                 print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
192                 return 2
193
194                 segmenter_actual = seg_name
195                 result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
196                 segmentation_ran = True
197                 if isinstance(result, Failure):
198                     print(f"[worker] segmenter FAILED: {result.message}")
199                     return 3
200
201                     segments = result.value if hasattr(result, "value") else result
202                     status = "success"
203                     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
204                     # empty match ? the cold-start failure mode (the substrate detector yielded no

```

```

usable
205         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
206         if not segments:
207             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
208                             or getattr(config.indexing, "detector_impl", None) or
"auto")
209             logger.warning(
210                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
211                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
212                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
213                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
214                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
215                 video_id, segmenter_actual, detector_impl,
216             )
217         finally:
218             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
219             emit_indexer_report(
220                 db=db, video_id=video_id, video_path=local_video, config=config,
221                 val_results=val_results, val_context=val_context,
222                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
223                 was_reingest=False, segmentation_ran=segmentation_ran,
224             )
225
226         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
227         out_local = os.path.join(scratch, "outputs", video_id)
228         os.makedirs(out_local, exist_ok=True)
229         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
230         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
231
232         out_prefix = args.out_uri.rstrip("/")
233         pushed = []
234         for fn in sorted(os.listdir(out_local)):
235             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
236             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
237             pushed.append(uri)
238
239         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
240         for u in pushed:
241             print("    ", u)
242
243         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
244         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
245         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
246         # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
247         if getattr(args, "done_uri", None):
248             _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
249         return 0
250
251
252         #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a

```

```

253     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
254     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
255
256
257     def _probe_video_fps_width(path: str):
258         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
259
260         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
261         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
262         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
263         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
264         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
265         rather than guesses.
266         """
267         import json
268         import subprocess
269         try:
270             out = subprocess.check_output(
271                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
272                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
273                 stderr=subprocess.DEVNULL).decode()
274             st = (json.loads(out).get("streams") or [{}])[0]
275             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
276             fps = float(num) / float(den or "1")
277             width = float(st.get("width") or 0)
278             if fps > 0 and width > 0:
279                 return fps, width
280             except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
281                 pass
282             return None
283
284
285     def _resolve_train_detector(args: argparse.Namespace, config: Any):
286         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
287         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
288         stub=CPU mock). Returns the runner, or prints an error + returns None.
289
290         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
291         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
292         no shuttle detector and is rejected.
293         """
294         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
295
296         seg_cls = segmenter_registry.get(args.segmenter)
297         if not seg_cls:
298             print(f"[worker] unknown segmenter: {args.segmenter!r} "
299                   f"(have {list(segmenter_registry.list_available())})")
300             return None
301         seg = seg_cls(None, config)
302         if not isinstance(seg, TrajectoryHybridSegmenter):
303             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
304                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
305             return None
306         try:

```



```

307         runner, _ = seg._resolve_runner(config.get("indexing", {}))
308     except NotImplementedError as e:
309         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
310         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
311         traceback.
312         print(f"[worker] detector not available: {e}")
313         return None
314     ok, msg = runner.healthcheck()
315     if not ok:
316         print(f"[worker] detector environment not ready: {msg}")
317         return None
318     print(f"[worker] detector ready ({args.segmenter}): {msg}")
319     return runner
320
321 def cmd_train(args: argparse.Namespace) -> int:
322     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
323     shell out
324     to training.gen0.harness (backend must NOT import training) -> push the artifact
325     bundle.
326     Two trajectory sources (the train pipeline + harness are identical either way):
327     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
328     synthetic
329     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
330     CI.
331     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
332     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
333     This
334     is the M3.5 "first real training" path; only the trajectory source flips.
335     """
336     import subprocess
337
338     config = load_config(args.config) if args.config else load_config()
339     _apply_overrides(config, args.set)
340     if not _run_startup_checks(config):
341         return 2
342     storage_cfg = config.storage.model_dump()
343
344     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
345     labels_dir = os.path.join(scratch, "labels")
346     traj_dir = os.path.join(scratch, "trajectories")
347     videos_dir = os.path.join(scratch, "videos")
348     os.makedirs(labels_dir, exist_ok=True)
349     os.makedirs(traj_dir, exist_ok=True)
350
351     corpus = args.corpus_uri.rstrip("/")
352     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
353     config=storage_cfg)
354         if u.lower().endswith(".csv"))
355     if len(label_uris) < 2:
356         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
357         {len(label_uris)} "
358         f"under {corpus}/labels/")
359         return 2
360
361     # Real-detector source: resolve the runner once + index the bucket's videos/ by
362     stem.
363     runner = None
364     video_by_stem: dict = {}
365     if args.trajectory_source == "detector":
366         os.makedirs(videos_dir, exist_ok=True)
367         runner = _resolve_train_detector(args, config)
368         if runner is None:
369             return 2

```

```

363         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
364             vname = storage.parse_uri(vuri).name
365             if not vname.lower().endswith(_VIDEO_EXTS):
366                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
367             video_by_stem[os.path.splitext(vname)[0]] = vuri
368
369         print(f"[worker] trajectory source: {args.trajectory_source}")
370         specs: List[dict] = []
371         for uri in label_uris:
372             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
373             name = fname.replace(".rallies.csv", "").replace(".csv", "")
374             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
375             if args.trajectory_source == "detector":
376                 vuri = video_by_stem.get(name)
377                 if not vuri:
378                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
379                     continue
380                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
381                                     config=storage_cfg)
382                 video_id = compute_video_id(local_video)
383                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
384                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
385                 meta = _probe_video_fps_width(local_video)
386                 if meta is None:
387                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
388                     continue
389                 fps, frame_width = meta
390                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
391                 if not traj:
392                     print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
393                     continue
394                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
395                             "fps": fps, "frame_width": frame_width})
396             else:
397                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
398                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
399                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
400                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
401                             "fps": args.fps, "frame_width": args.frame_width})
402
403         if len(specs) < 2:
404             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
405             return 2
406         manifest_path = os.path.join(scratch, "manifest.json")
407         with open(manifest_path, "w", encoding="utf-8") as f:
408             json.dump(specs, f, indent=2)
409         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
410         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
411
412         out_dir = os.path.join(scratch, "artifacts")
413         cmd = [sys.executable, "-m", "training.gen0.harness",
414               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]

```

```

415         if args.floor:
416             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json")),
config=storage_cfg)
417                 if "://" in args.floor else args.floor)
418             cmd += ["--floor", floor_local]
419         print("[worker] training:", " ".join(cmd))
420         rc = subprocess.run(cmd).returncode
421         if rc != 0:
422             print(f"[worker] gen0 harness failed (rc={rc})")
423             return rc
424
425         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
426         bundle = os.path.join(out_dir, args.version)
427         out_prefix = args.out_uri.rstrip("/")
428         pushed = []
429         for fn in sorted(os.listdir(bundle)):
430             uri = f"{out_prefix}/weights/{args.version}/{fn}"
431             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
432             pushed.append(uri)
433         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
434         for u in pushed:
435             print("    ", u)
436         return 0
437
438
439     def build_parser() -> argparse.ArgumentParser:
440         p = argparse.ArgumentParser(prog="python -m backend.worker",
441                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
442         p.add_argument("-v", "--verbose", action="store_true",
443                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
444         sub = p.add_subparsers(dest="cmd", required=True)
445
446         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
447         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
448         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
449         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
450         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
451         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
452         pi.add_argument("--max-frames", type=int, default=None)
453         pi.add_argument("--done-uri", default=None,
454                       help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
455                       "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
456                       "Default-OFF: omit it for the normal local run.")
457         pi.add_argument("--set", action="append", default=[], metavar="k=v",
458                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
459         pi.set_defaults(func=cmd_infer)
460
461         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
462         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
463                       "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
464         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
465         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
466         pt.add_argument("--trajectory-source", choices=["synth", "detector"],

```

```

default="synth",
467             help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
468             "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
469             "(needs videos/ in the corpus; honours detector_impl).")
470     pt.add_argument("--segmenter", default="wasb_hybrid",
471                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
472     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
473     pt.add_argument("--config", default=None)
474     pt.add_argument("--scratch", default=None)
475     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
476     pt.add_argument("--frame-width", type=float, default=1280.0,
477                     help="synth trajectory frame width (detector probes the video)")
478     pt.add_argument("--set", action="append", default=[], metavar="k=v")
479     pt.set_defaults(func=cmd_train)
480     return p
481
482
483     def _setup_logging(verbose: bool) -> None:
484         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
485         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
486         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
487         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
488         entry), not at import, so importing the module never reconfigures a caller's
logging."""
489         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
490         # configured logging at import), which would leave the level at WARNING and re-
suppress the
491         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
492         logging.basicConfig(
493             level=logging.DEBUG if verbose else logging.INFO,
494             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
495             datefmt="%H:%M:%S",
496             force=True,
497         )
498
499
500     def main(argv=None) -> int:
501         args = build_parser().parse_args(argv)
502         _setup_logging(getattr(args, "verbose", False))
503         return args.func(args)
504
505
506     if __name__ == "__main__":
507         sys.exit(main())
508

```

13. user (2026-06-21T10:45:32.385Z)

```

1     # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2     #
3     # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4     # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the

```

```

owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40         torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45     # --- Main application env ---
46     WORKDIR /app
47     COPY pyproject.toml README.md ./
48     COPY backend ./backend
49     COPY training ./training
50     RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at

```

```

53     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54     ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55         WASB_REPO_DIR=/opt/models/WASB-SBDT \
56         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57         RALLY_DETECTOR_IMPL=native
58
59     # The dispatcher (CloudRunCompute) supplies the per-execution args:
60     #   infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61     ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

14. user (2026-06-21T10:45:32.402Z)

```

1     # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3     The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4     dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5     a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7     > Status: the dispatch shim + image are landed + mock-tested (M6). The real
build + push +
8     > run on an L4 is the M7 owner step, within the $100/?10k budget (D17) ?
confirm cmd + hourly
9     > cost first, delete every resource after ([[gcp-vm-always-delete]]). CI does
not build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run Job (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29         --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30         --max-retries 0 --task-timeout 3600 \
31         --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```
34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the cloud-serving profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:

```

```

38     ``bash
39     export DEPLOYMENT_PROFILE=cloud-serving
40     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44     ``
45
46     ## 4. M7 acceptance (the owner runlog)
47     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52     ## Gotchas
53     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57     - A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times
out
58     (`ExecutionPolicy.max_wait_sec`). A failure-marker is a follow-up.
59     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
60

```

15. assistant (2026-06-21T10:45:35.237Z)

Now let me inspect the storage layer and execution_policy to verify the integration contracts (parse_uri/get_storage/fetch signatures, ExecutionPolicy.poll_until behavior).

16. user (2026-06-21T10:45:36.198Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    import tempfile
17    from typing import Any, Dict, Iterator, Optional
18

```

```

19 from backend.storage.base import StorageBackend
20 from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22 __all__ = [
23     "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24     "get_storage", "fetch", "put", "list_uris",
25     "acquire_local_input", "input_is_local", "resolve_input_ref",
26 ]
27
28
29 def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31     if storage_config is None:
32         return {}
33     if hasattr(storage_config, "model_dump"):
34         return storage_config.model_dump() # type: ignore[no-any-return]
35     return dict(storage_config)
36
37
38 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39     """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40     config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41     callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45     sc = _storage_dict(storage_config)
46     uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                             input_root=sc.get("input_root", ""))
48     return parse_uri(uri)
49
50
51 def input_is_local(raw: str, storage_config: Any = None) -> bool:
52     """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53     plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54     that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55     ``resolve_input_ref``)."""
56     return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59 def acquire_local_input(raw: str, storage_config: Any = None, *,
60                         scratch_parent: "str | None" = None) -> str:
61     """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63     Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64     byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65     non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66     path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67     regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68     worker's ``cmd_infer`` already ships."""
69     sc = _storage_dict(storage_config)
70     ref = resolve_input_ref(raw, sc)

```



```

71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):

```

```
126         yield r.uri
127
```

17. user (2026-06-21T10:45:36.780Z)

```
1      """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3      Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4      - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5      for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6      so it is abandoned, not joined ? it never blocks the process.
7      - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8      ``max_wait_sec`` deadline).
9
10     **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17     from __future__ import annotations
18
19     import logging
20     import threading
21     import time as _time
22     from dataclasses import asdict, dataclass
23     from enum import Enum
24     from typing import Callable, Optional, TypeVar
25
26     T = TypeVar("T")
27
28     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29     LOG = logging.getLogger("execution.policy")
30
31
32     class WaitMode(str, Enum):
33         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35         ABORT = "abort" # one attempt; on hang/fail, stop immediately
36         BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46         mode: WaitMode = WaitMode.WAIT_AND_RESUME
47         poll_every_n_sec: float = 10.0
```

```

48         op_timeout_sec: float = 120.0          # HARD per-attempt deadline ? the hang-killer
that was missing
49         max_wait_sec: float = 1800.0          # total wall budget for a poll loop
50         max_retries: int = 5                  # retries (=> max_retries+1 attempts) for
failures/hangs
51         backoff_base_sec: float = 3.0          # exponential: backoff_base * 2**attempt
52         on_timeout: str = "reschedule"         # reschedule | partial | fail (consumed by
P2/P3)
53         resumable: bool = True
54
55         def to_dict(self) -> dict:
56             d = asdict(self)
57             d["mode"] = self.mode.value
58             return d
59
60         @classmethod
61         def from_dict(cls, d: dict) -> "ExecutionPolicy":
62             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63             kw = {k: v for k, v in d.items() if k in known}
64             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                 kw["mode"] = WaitMode(kw["mode"])
66             return cls(**kw)
67
68
69         #: The sane default (also what a job gets if it declares no policy).
70         DEFAULT_POLICY = ExecutionPolicy()
71
72
73         def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74             """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75             return policy.backoff_base_sec * (2 ** attempt)
76
77
78         def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79             label: str = "op", logger: Optional[logging.Logger] = None,
80             sleep: Callable[[float], None] = _time.sleep) -> T:
81             """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
82
83             Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
84             last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
85             """
86             log = logger or LOG
87             attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
88             last_exc: Optional[BaseException] = None
89             for attempt in range(attempts):
90                 box: dict = {}
91                 done = threading.Event()
92
93                 # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
94                 # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95                 def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96                     try:
97                         _box["val"] = fn()
98                     except BaseException as e:          # relay ANY error (incl. timeouts) to the
caller thread
99                         _box["exc"] = e
100                     finally:
101                         _done.set()
102
103                 threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()

```

```

104         if done.wait(timeout=policy.op_timeout_sec):
105             if "exc" in box:
106                 last_exc = box["exc"]
107                 log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108             else:
109                 if attempt:
110                     log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111                     return box["val"]                # type: ignore[return-value]
112             else:
113                 last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114                 log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                             policy.op_timeout_sec)
116                 if attempt < attempts - 1:
117                     wait = _backoff_sec(policy, attempt)
118                     log.debug("%s: backing off %.1fs before retry", label, wait)
119                     sleep(wait)
120                 assert last_exc is not None
121                 log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122                 raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                   label: str = "op", logger: Optional[logging.Logger] = None,
127                   sleep: Callable[[float], None] = _time.sleep,
128                   monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                 label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

18. assistant (2026-06-21T10:45:40.586Z)

Now let me check the StorageRef for ``.exists``/``.backend`` attributes, and verify `parse_uri` handling of local paths (since `cloud_run.py` calls `get_storage(ref.backend).exists(ref)`` and the tests use local `tmp_path`` URIs without a `local://`` scheme).

19. user (2026-06-21T10:45:41.949Z)

```
1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs       (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
```

```

57         return StorageRef("local", "", s, f"local://{s}")
58     scheme = m.group(1).lower()
59     rest = s[m.end():]
60     if scheme == "gs":
61         scheme = "gcs"
62     if scheme in ("local", "gdrive"):
63         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80         ``"gcs://bucket/x"``).
81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
84         path used today, preserving byte-for-byte behaviour.
85         ""
86         s = str(raw)
87         if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
92         if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                    # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}://{s}"
98         return s                            # 5. bare/relative local path ? unchanged
99

```

20. user (2026-06-21T10:45:42.010Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted

```

```

Drive ?
6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      ""
8      from __future__ import annotations
9
10     import os
11     from abc import ABC, abstractmethod
12     from typing import Iterator
13
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class StorageBackend(ABC):
18         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19         worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21         @abstractmethod
22         def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24             """Make the object available as a local file; return its local path. For the
25             ``local`` backend this is an identity passthrough (the object is already
26             local)."""
27
28         @abstractmethod
29         def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
30                 content_type: "str | None" = None) -> StorageRef:
31             """Upload/copy a local file to ``ref``; return ``ref``."""
32
33         @abstractmethod
34         def exists(self, ref: StorageRef) -> bool: ...
35
36         @abstractmethod
37         def stat(self, ref: StorageRef) -> StorageStat: ...
38
39         @abstractmethod
40         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
41             """Yield a ``StorageRef`` per object under ``prefix``."""
42
43         @abstractmethod
44         def delete(self, ref: StorageRef) -> None: ...
45
46         def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
47             """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
48             sign (``local`` cannot; Drive only emulates) ? callers must treat
49             ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
50             raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")

```

21. assistant (2026-06-21T10:45:45.200Z)

Now let me run the tests and the lint/type checks the CI would run, to ground any hygiene findings. Let me check whether ruff/mypy are configured and run the new test file.

22. user (2026-06-21T10:45:55.265Z)

```

..... [100%]
9 passed in 5.93s

```

23. user (2026-06-21T10:45:56.184Z)

```

All checks passed!
---MYPY---

```

Success: no issues found in 3 source files

24. assistant (2026-06-21T10:46:02.176Z)

Tests pass, ruff and mypy are clean. Now let me verify a few specific integration concerns. First, the local-storage `exists`/`put` path with bare local paths (the test uses `str(tmp\_path / "bucket")` without a `local://` scheme), and the GCS backend `put` to confirm `gs://`/`gcs://` round-trips correctly. Let me also check how `gs://` URIs flow: the worker writes the marker to `done\_uri` (gs://) and the dispatcher reads `gs://...`. Let me confirm GCS storage and local storage `put`/`fetch`/`exists`.

25. user (2026-06-21T10:46:03.189Z)

```
1      """Local-filesystem storage backend ? the default, zero new dependencies.
2
3      Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4      file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5      the whole pipeline behaves exactly as before the storage layer existed.
6      """
7      from __future__ import annotations
8
9      import os
10     import shutil
11     from datetime import datetime
12     from typing import Iterator
13
14     from backend.storage.base import StorageBackend
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class LocalStorage(StorageBackend):
19         def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20             # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21             self.base_dir = base_dir
22
23         def _path(self, ref: StorageRef) -> str:
24             p = ref.key
25             if self.base_dir and not os.path.isabs(p):
26                 return os.path.join(self.base_dir, p)
27             return p
28
29         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30             # The object is already on the local FS ? return its path (identity). ``dst`` is
31             # honoured only if it names a *different* location (a copy is then made).
32             src = self._path(ref)
33             if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                 return src
35             dst = str(dst)
36             if os.path.exists(dst) and not overwrite:
37                 return dst
38             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39             shutil.copyfile(src, dst)
40             return dst
41
42         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43             dst = self._path(ref)
44             if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                 os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                 shutil.copyfile(str(local_path), dst)
47             return ref
48
49         def exists(self, ref) -> bool:
50             return os.path.exists(self._path(ref))
```



```

51
52     def stat(self, ref) -> StorageStat:
53         st = os.stat(self._path(ref))
54         return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56     def list(self, prefix) -> Iterator[StorageRef]:
57         base = self._path(prefix)
58         if os.path.isdir(base):
59             for root, _dirs, files in os.walk(base):
60                 for fn in sorted(files):
61                     full = os.path.join(root, fn)
62                     yield StorageRef("local", "", full, f"local://{full}")
63         elif os.path.exists(base):
64             yield StorageRef("local", "", base, f"local://{base}")
65
66     def delete(self, ref) -> None:
67         path = self._path(ref)
68         if os.path.exists(path):
69             os.remove(path)
70

```

26. user (2026-06-21T10:46:03.394Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional
12
13    from backend.storage.base import StorageBackend
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class GCSStorage(StorageBackend):
18        def __init__(self, *, client: Any = None, project: Optional[str] = None,
19            signed_url_ttl: int = 3600, **_ignored) -> None:
20            self._ttl = signed_url_ttl
21            if client is not None:
22                self._client = client
23            return
24        from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25        self._client = storage.Client(project=project)
26
27        def _blob(self, ref: StorageRef):
28            return self._client.bucket(ref.bucket).blob(ref.key)
29
30        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31            if dst is None:
32                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33            import os
34            dst = str(dst)
35            if os.path.exists(dst) and not overwrite:
36                return dst
37            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)

```

```

38         self._blob(ref).download_to_filename(dst)
39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

27. assistant (2026-06-21T10:46:11.459Z)

Now I have a clear picture of the storage flow. Let me focus on a subtle integration concern. In ``cloud_run.py``, ``_read_json`` calls ``fetch(uri, config=self._storage_config)`` with ``dst=None``. For local backend this is identity passthrough (returns the path). But for GCS, ``fetch_to_local`` raises ``ValueError`` when ``dst`` is ``None``. This means the dispatcher's ``_read_json`` for ``gs://`` URIs in production would always hit the ``except Exception: return {}`` ? silently swallowing the report and the marker read.

Let me verify this is the real behavior. The ``_read_marker`` calls ``_read_json(done_uri)`` where ``done_uri`` is ``gs://.../_dispatch/<token>.done``, and ``_read_json`` calls ``fetch(uri, config=...)`` with no ``dst``. For GCS that raises ``ValueError`` ? caught ? returns ``{}``. Let me confirm ``fetch`` signature default for ``dst``.

The ``fetch`` helper: ``def fetch(uri, dst=None, ...)`` ? ``get_storage(ref.backend).fetch_to_local(ref, dst, ...)``. For GCS, ``fetch_to_local`` with ``dst=None`` raises ``ValueError``. Confirmed.

Let me check whether there's a guard / whether this path is reachable, and verify the marker-read consequence carefully.

28. user (2026-06-21T10:46:13.759Z)

```

126     # --- storage-backed completion signal (bucket-poll) ---
127     def _read_marker(self, done_uri: str) -> Optional[dict]:
128         """The done-marker dict if the worker has written it, else None ? poll_until
129         keeps waiting."""
130         ref = parse_uri(done_uri)
131         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
132             return None
133         return self._read_json(done_uri) or {} # empty dict still terminates the poll
134         (present == done)
135
136     def _read_json(self, uri: str) -> dict:
137         try:
138             local = fetch(uri, config=self._storage_config)

```

```

137         with open(local, "r", encoding="utf-8") as f:
138             data: Any = json.load(f)
139             return data if isinstance(data, dict) else {}
140     except Exception:
141         return {}
142

```

29. assistant (2026-06-21T10:46:25.425Z)

This is a real, grounded integration bug. Let me trace the production (`gs://`) consequence precisely:

1. `_read_marker(done_uri)` where `done_uri = "gs://bucket/_dispatch/<tok>.done"`.
2. `exists(ref)` ? GCS exists check works (returns True once worker writes marker).
3. `_read_json(done_uri)` ? `fetch(uri, config=...)` with `dst=None` ? GCS `fetch_to_local(ref, None)` ? `**raises ValueError("GCSStorage.fetch_to_local requires a dst path")**` ? caught by `except Exception` ? returns `{}`.
4. Back in `_read_marker`: `self._read_json(done_uri) or {}` ? `{}`. Since `{}` is non-None, `poll_until` terminates (present == done).
5. In `run_infer`: `marker = {}` ? `video_id = str(marker.get("video_id", ""))` ? `""` ? `outputs_uri = ""` ? `report = {}` ? `rc = int(marker.get("returncode", 0))` ? `0`.

So in production with `gs://`, the dispatcher returns `ComputeResult(returncode=0, outputs_uri="", video_id="", report={})` ? it loses the `video_id`, `outputs_uri`, and the real `returncode` (always reports 0 even if the worker returned non-zero). The tests pass only because they use `**local**` URIs where `fetch` is an identity passthrough that returns the path. This is exactly the kind of "tests use local, prod uses gs://" gap the lens asks about.

Let me verify there's no other `fetch` overload or default-dst handling I'm missing, and confirm the worker's `put` side works for `gs://` (it passes a real local path, so that's fine). Let me double-check that no `dst` is auto-generated anywhere in `fetch`.

The `fetch` helper clearly passes `dst=None` straight through. Confirmed bug.

Now let me check the other lens items. Let me verify circular imports by importing the modules cleanly and confirming `google-cloud-run` is truly lazy.

30. user (2026-06-21T10:46:27.691Z)

```

google.cloud.run loaded? False
run_v2 in modules? False
import OK, no cycle

```

31. assistant (2026-06-21T10:46:38.741Z)

No import cycle, and `google-cloud-run` is genuinely lazy (not imported on module import). Good. Note `GoogleCloudRunJobClient()` is instantiated eagerly in `get_compute_backend` when `run_client` is None, but its `__init__` is the default (no import), and `run_v2` is imported only inside `run_job`. So even constructing the real client doesn't pull the SDK ? only an actual dispatch does. That's fine.

Now let me verify the `config_overrides` tuple ordering concern (item 3) and the `--set` handling. In `cloud_run.py` line 109: `for kv in (*spec.config_overrides, *self._container_overrides)`. User overrides first, then the inline-forcing overrides last. In the worker's `_apply_overrides`, later `--set` wins (sequential `setattr`). So `compute_target=local_gpu` and `detector_impl=native` correctly override any user attempt. The test confirms this. Good.

Now, one concern: in `_dispatch_remote` (worker), it builds `ComputeJobSpec(config_overrides=tuple(args.set or ()))`. But `args.set` defaults to `[]` (from `argparse default=[]`). The dispatch path passes the user's `--set` through to the spec, which `CloudRunCompute` then appends the inline-forcing overrides to. Good.

Let me check the `run_v2` type:ignore and the `getattr` return type on line 77 ? mypy passed, so

that's clean. Let me verify one more thing: the README says `gs://` but the `GoogleCloudRunJobClient.run\_job` return expression. Line 77: `return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or "execution"`. This is `pragma: no cover`. If `op.metadata` exists but `.name` is empty string, `""` or "execution" ? "execution". Fine. mypy passed.

Now let me verify the Dockerfile COPY paths exist (pyproject.toml, README.md, backend, training dirs) and the pyproject install.

32. user (2026-06-21T10:46:40.446Z)

```
README.md
backend
pyproject.toml
training
---
5:# reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
16:[project]
17:name = "badminton-highlight-indexer"
18:version = "0.1.0"
23:authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
85:# `backend` is a namespace package (no __init__.py); list both import roots
88:packages = ["backend", "training"]
```

33. assistant (2026-06-21T10:46:44.307Z)

All COPY source paths exist (`pyproject.toml`, `README.md`, `backend`, `training`), and `pyproject.toml` packages both `backend` and `training`. Let me verify the WASB env steps in the Dockerfile against `gpu\_setup.sh` (item 5 ? faithfulness).

34. user (2026-06-21T10:46:46.085Z)

```
3:# Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
5:# *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6:# main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-installed
7:# with the indexer's torch (framework conflict).
11:# ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is NOT
18:WASB_REPO="$MODELS_DIR/WASB-SBDT"
29:# Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
35:echo "[gpu] [2/5] WASB-SBDT repo"
36:if [ ! -d "$WASB_REPO/.git" ]; then
37:  git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
40:echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB stack)"
41:conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
43:python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
44:  --extra-index-url https://download.pytorch.org/whl/cu113
45:[ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
46:# WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it omits,
so
47:# `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.) numpy
is
48:# pinned <1.24 for torch-1.11 ABI compatibility.
49:python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm
pyyaml matplotlib "numpy<1.24" || true
51:echo "[gpu] [4/5] pretrained badminton weights"
52:WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
61:import torch
62:ok = torch.cuda.is_available()
63:print("[gpu]      torch", torch.__version__, "| cuda_available", ok,
64:      "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
68:echo "[gpu] native WASB env ready.  Activate with:  source $CONDA_DIR/etc/profile.d/conda.sh
&& conda activate wasb"
```

35. assistant (2026-06-21T10:46:58.511Z)

The Dockerfile faithfully mirrors `gpu\_setup.sh`: same conda env (py3.8), same torch 1.11.0+cu113 / torchvision 0.12.0+cu113, same WASB-SBDT clone, same supplementary deps (`hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib "numpy<1.24"`), same TOS-accept dance, same `WASB\_PYTHON`/`WASB\_REPO\_DIR`/`WASB\_WEIGHTS` env. The weights are intentionally not baked (WASB-C3), staged at runtime ? consistent with the README. No build-breaker in the COPY/install steps.

One faithfulness detail: `gpu\_setup.sh` uses `opencv-python-headless` and the Dockerfile matches it. Good.

Now let me double-check one subtle thing about the worker `\_run\_startup\_checks` interaction on the container side. The dispatched container is forced to `compute\_target=local\_gpu`. But the README/Dockerfile says the container runs with the cloud-serving profile env. Actually no ? the container's args force `compute\_target=local\_gpu` via `--set`, but `deployment.profile` is NOT set by the dispatcher. Let me verify the container doesn't accidentally re-dispatch. `get\_compute\_backend` checks `compute\_target`, which is forced to `local\_gpu` ? returns None ? inline. Correct, no re-dispatch regardless of profile. Good.

Let me verify the failure-marker gap is real and acknowledged (it is ? README "Gotchas" + worker comment both note a failed job writes no marker). That's a documented known limitation, not a hidden bug ? I'll note it as minor since the dispatcher's poll times out rather than reporting failure, but it's explicitly called out as a follow-up. Given the contract says "completion = bucket done-marker," and failure handling is explicitly deferred, I'll treat it as a documented limitation (nit-level at most, already disclosed).

Let me now confirm the gs:// `\_read\_json` bug isn't somehow mitigated by a tempfile default in any wrapper. Let me grep for any default-dst behavior.

36. user (2026-06-21T10:46:59.442Z)

```
backend\storage\local.py:29: def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-30- # The
object is already on the local FS ? return its path (identity). ``dst`` is
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-31- #
honoured only if it names a *different* location (a copy is then made).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-32- src =
self._path(ref)
backend\storage\local.py-33- if dst is None or os.path.abspath(str(dst)) ==
os.path.abspath(src):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-34-
return src
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py-35- dst =
str(dst)
--
backend\storage\gcs.py:30: def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py-31- if dst is None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-32- raise
ValueError("GCSStorage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-33- import os
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-34- dst =
str(dst)
backend\storage\gcs.py-35- if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py-36- return
dst
--
backend\storage\gdrive.py:112: def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py-113- # The file is already local (via the mount): no dst =>
identity passthrough to the mount
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-114- #
path; a dst => copy out (a byte-stable scratch copy), exactly as the local backend does.
```

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-115-
self._require_present(ref)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-116-           return
self._local.fetch_to_local(self._local_ref(ref), dst, overwrite=overwrite)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py-117-
backend\storage\gdrive.py-118-     def put(self, local_path, ref, *, content_type=None) ->
StorageRef:
--
backend\storage\base.py:22:     def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend\storage\base.py-23-                                     *, overwrite: bool = False) -> str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-24-         """Make
the object available as a local file; return its local path. For the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-25-         ``local``
backend this is an identity passthrough (the object is already local)."""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-26-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py-27-
@abstractmethod
backend\storage\base.py-28-     def put(self, local_path: "str | os.PathLike[str]", ref:
StorageRef, *,
--
backend\storage\s3.py:38:     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py-39-         if dst is None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-40-             raise
ValueError("S3Storage.fetch_to_local requires a dst path")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-41-             dst =
str(dst)
backend\storage\s3.py-42-             if os.path.exists(dst) and not overwrite:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-43-                 return
dst
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py-44-
os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)

```

37. assistant (2026-06-21T10:47:16.463Z)

This confirms the bug is real for BOTH cloud backends: `GCSStorage.fetch\_to\_local` AND `S3Storage.fetch\_to\_local` raise `ValueError` when `dst` is `None`. The dispatcher's `\_read\_json` calls `fetch(uri, config=...)` with no `dst`, so for any real cloud bucket (the entire reason CloudRunCompute exists), the marker read and report read silently fail and return `{}`. The local-URI tests can't catch this because local is an identity passthrough.

This is a blocker for the production path: in prod, `run\_infer` would always return `ComputeResult(returncode=0, video\_id="", outputs\_uri="", report={})` regardless of what the worker actually did ? masking worker failures (always rc=0) and losing the outputs location. The fix is for `\_read\_json` to pass an explicit `dst` (e.g. a temp file) to `fetch`.

The contract note says "the real client is owner-verified on M7 (CI uses a fake)" ? but this bug is in `CloudRunCompute` itself (the storage read), NOT in the `CloudRunJobClient`. It is exercised by the real GCS storage backend, which is not the fake-client concern. So M7 owner verification would be the first time this surfaces ? and it would silently report success with empty results, which is a particularly nasty failure mode (no exception, just wrong data).

Let me make sure I have the line numbers exact for the finding and verify there are no other groundable issues. Let me re-examine `\_read\_marker`: line 132 `self.\_read\_json(done\_uri) or {}`. For local this returns the real marker. For gs:// it returns `{}` (from the swallowed `ValueError`), and `{}` or `{}` ? `{}`, terminating the poll with an empty marker. Confirmed.

I have enough grounded findings. Let me compile the report. Summary of what I verified clean: no import cycle, lazy google-cloud-run, config_overrides tuple ordering correct (inline-forcing wins), factory shape mirrors storage, Dockerfile COPY paths exist + WASB steps faithful to gpu_setup.sh, ruff+mypy clean, all 9 tests pass.

38. user (2026-06-21T10:47:33.528Z)

Structured output provided successfully

39. assistant (2026-06-21T10:47:46.699Z)

I've completed the INTEGRATION / HYGIENE / CONVENTIONS review of the M6 change. Summary of findings:

****1 blocker**** ? ``backend/compute/cloud_run.py:136``: ``_read_json`` calls ``fetch()`` with no ``dst``. Local backend treats this as an identity passthrough (so all 9 tests pass on ``tmp_path`` URIs), but ``GCSStorage``/``S3Storage.fetch_to_local`` both ``raise ValueError`` when ``dst`` is ``None``. The bare ``except Exception: return {}`` swallows it, so every real ``gs:///``/``s3:///`` dispatch returns ``ComputeResult(returncode=0, video_id="", outputs_uri="", report={})`` ? masking worker failures as success and losing the outputs path. This is ``CloudRunCompute``'s own storage read against the real backend, not the fake-client seam M7 covers, so it would silently surface (wrong data, no exception) on the M7 owner run.

****1 nit**** ? failed container jobs write no done-marker, so the dispatcher times out instead of reporting the real rc. Already explicitly documented as a follow-up in both the worker comment and README Gotchas; flagged only so it isn't lost.

****Verified clean**** (no findings):

- No import cycle; ``import backend.compute / backend.worker / backend.compute.cloud_run`` all succeed and ``google.cloud.run_v2`` is NOT in ``sys.modules`` afterward ? truly lazy (imported only inside ``run_job``; even constructing ``GoogleCloudRunJobClient()`` doesn't pull it).
- ``config_overrides`` tuple ordering correct: user overrides emitted first, inline-forcing ``compute_target=local_gpu`` + ``detector_impl=native`` last, and the worker's sequential ``setattr`` makes last-wins ? so the container can't be tricked into re-dispatching. Test covers this.
- Factory/lazy idiom mirrors ``backend/storage`` (``get_compute_backend`` ? ``None`` default-OFF; lazy ``cloud_run`` import only on ``cloud_run`` target).
- Dockerfile: COPY sources (``pyproject.toml``, ``README.md``, ``backend``, ``training``) all exist; ``pyproject.toml`` packages both roots; WASB env steps are faithful to ``deploy/digitalocean/gpu_setup.sh`` (same py3.8 conda env, torch 1.11.0+cu113 / torchvision 0.12.0+cu113, WASB-SBDT clone, supplementary deps, ``numpy<1.24``, the three ``WASB_*`` envs; weights intentionally not baked per WASB-C3).
- ``ruff check`` and ``mypy`` both clean on the new code; all 9 tests in ``tests/test_compute_backend.py`` pass.